

1. Crear el realm piedrazul

En el panel principal de Keycloak (asumo que entraste como admin al realm master):

1. Arriba a la izquierda, donde dice **"Keycloak"** (o **"master"**) hay un dropdown. Desplégalo.
2. Click en **"Create realm"**.
3. Llena el formulario:
 - **Realm name:** piedrazul
 - **Enabled:** ON (debería estar por defecto)
 - **Resource file:** déjalo vacío (no estamos importando).
4. Click en **"Create"**.

Después de crear, asegúrate de que arriba a la izquierda el dropdown ahora muestra piedrazul. **Todo lo que sigue se hace dentro de este realm**, no en master.

2. Configurar el realm (tokens y sesiones)

Menú izquierdo → **Realm settings**.

2.1 Pestaña General

- **Display name:** Pie Drazul (lo que verá el usuario si algún día usas la pantalla de login de Keycloak).
- **HTML Display name:** `<b>Pie Drazul</b>` (opcional).
- Resto, déjalo como está.
- Click en **Save** al final de la pestaña.

2.2 Pestaña Tokens

Estos valores son los que más impactan UX y seguridad. Te recomiendo estos para desarrollo:

Campo	Valor recomendado	Notas
Default Signature Algorithm	RS256	Asimétrico → el gateway valida con la clave pública (JWKS). No compartas secretos.
Access Token Lifespan	15 minutes	El token corto es seguro; el refresh token cubre la sesión larga.
Client Session Idle	30 minutes	

Campo	Valor recomendado	Notas
Client Session Max	10 hours	
SSO Session Idle	30 minutes	
SSO Session Max	10 hours	
Refresh Token Max Reuse	0	Refresh token rotation: el usado se invalida al refrescar.
Revoke Refresh Token	ON	Necesario para que "Max Reuse=0" tenga efecto.

Click **Save** al final.

2.3 Pestaña Login

- **User registration:** OFF (los usuarios los crea el usuarios-service, no el formulario de Keycloak).
- **Forgot password:** OFF por ahora.
- **Remember me:** OFF.
- **Email as username:** OFF (tú usas username separado de email).
- **Login with email:** OFF (mismo motivo).

Click **Save**.

3. Crear los roles del realm

Menú izquierdo → **Realm roles** → botón **Create role**.

Crea **uno por uno** estos cuatro roles (los mismos nombres que ya tienes en DataInitializer.java):

Role name	Description
ADMINISTRADOR	Acceso total al sistema
AGENDADOR	Crea y gestiona citas en nombre de pacientes
PACIENTE	Autoservicio de citas propias
MEDICO_TERAPISTA	Gestiona su agenda y atenciones

Para cada uno: **Role name** → escribe el nombre exacto en mayúsculas → **Save**.

*Importante: respeta mayúsculas. Spring Security va a anteponer el prefijo **ROLE_** al convertirlos en **authorities** (**ROLE_ADMINISTRADOR**, etc.). Esto lo cableamos cuando hagamos el código del gateway.*

4. Crear el cliente piedrazul-frontend (JavaFX)

Menú izquierdo → **Clients** → botón **Create client**.

4.1 Step 1 — General settings

- **Client type:** OpenID Connect
- **Client ID:** piedrazul-frontend
- **Name:** Pie Drazul Frontend (JavaFX)
- **Description:** Cliente nativo de escritorio JavaFX
- Click **Next**.

4.2 Step 2 — Capability config

- **Client authentication:** **OFF** ← clave: lo dejamos **público** porque JavaFX no puede guardar un secret.
- **Authorization:** OFF.
- **Authentication flow:**
 - ☒ **Standard flow** (Authorization Code) — lo dejamos activo para el futuro
 - ☒ **Direct access grants** — **este es el que usaremos ahora desde JavaFX**
 - ☒ **Implicit flow**
 - ☒ **Service accounts roles** (necesita Client authentication ON)
- Click **Next**.

4.3 Step 3 — Login settings

Como vamos a usar Direct Access Grants y NO el flujo basado en redirección desde navegador, estos campos casi no se usan ahora, pero llénalos por si más adelante migras a PKCE:

- **Root URL:** deja vacío
- **Home URL:** deja vacío
- **Valid redirect URIs:** `http://localhost:*` (para futuro PKCE con loopback de JavaFX)
- **Valid post logout redirect URIs:** `http://localhost:*`

- **Web origins:** + (significa: "los mismos que valid redirect URIs", suficiente para que CORS no se queje si en algún momento agregas frontend web).
- Click **Save**.

4.4 Verificación rápida del cliente creado

Después de guardar, te lleva a la vista del cliente. Confirma en la pestaña **Settings**:

- "Access settings" → "Client authentication" = OFF.
- "Capability config" → "Direct access grants" = ON.

5. Mapeo de roles al token (paso crítico)

Por defecto Keycloak ya incluye los realm roles en el access token bajo la ruta realm_access.roles. Pero hay que confirmarlo, porque Spring Security va a leerlos justo de ahí.

Dentro del cliente piedrazul-frontend:

1. Pestaña **Client scopes**.
2. Verás una lista. Click en **piedrazul-frontend-dedicated** (el scope dedicado de este cliente).
3. Pestaña **Scope**: verifica que **Full scope allowed** esté ON (es lo default). Esto hace que los realm roles del usuario se mapeen al token sin restricción.
4. Pestaña **Mappers**: déjala vacía por ahora; el mapper realm roles ya viene heredado del scope roles que está asignado por defecto.

Para verificar que el scope roles está asignado:

- Vuelve a la pestaña **Client scopes** del cliente.
- Confirma que aparece **roles** con Assigned type = Default. Si no aparece, click en **Add client scope** → marca roles → **Add** → **Default**.

6. Crear un usuario de prueba

Menú izquierdo → **Users** → **Create new user**.

6.1 Datos del usuario

- **Username:** admin.test
- **Email:** admin.test@piedrazul.local
- **Email verified:** ON (para evitar el flujo "verifica tu email" en pruebas).
- **First name:** Admin
- **Last name:** Test

- **Enabled:** ON
- Click **Create**.

6.2 Establecer contraseña

Una vez creado, te lleva a la vista del usuario:

1. Pestaña **Credentials** → **Set password**.
2. **Password:** Admin.123 (o lo que prefieras, respetando tu política).
3. **Password confirmation:** lo mismo.
4. **Temporary:** **OFF** (si lo dejas ON, el usuario tendría que cambiar la pass en el primer login — y desde Direct Access Grants eso no funciona bien).
5. **Save password** → confirma en el diálogo.

6.3 Asignar el rol ADMINISTRADOR

1. Pestaña **Role mapping** → **Assign role**.
2. En el dropdown "Filter by", deja **"Filter by realm roles"**.
3. Marca ADMINISTRADOR → **Assign**.

Crea también, con el mismo procedimiento, al menos **un usuario por cada rol** para que puedas probar autorizaciones cuando hagamos el gateway:

Username	Password	Realm role
agendador.test	Agenda.123	AGENDADOR
paciente.test	Paciente.123	PACIENTE
medico.test	Medico.123	MEDICO_TERAPISTA

7. Probar que todo está bien (sin tocar código todavía)

Y se crearon mappers:

Necesitamos que el JWT incluya un claim usuario_id con tu UUID de dominio. Esto se hace en **dos partes**:

Parte 1: Definir el atributo a nivel de realm

En el panel de Keycloak, dentro del realm piedrazul:

1. Menú izquierdo → **Realm settings** → pestaña **User profile**.
2. Click en **Create attribute**.
3. Llena:
 - **Attribute (Name)**: usuario_id
 - **Display name**: Usuario ID (dominio)
 - **Multivalued**: OFF
 - **Enabled when scope**: deja vacío (siempre habilitado)
 - **Required**: OFF (lo dejamos opcional para no romper usuarios viejos; tu app garantizará setearlo siempre).
 - **Permissions**:
 - **Who can edit?**: marca **Admins** (no user, para que el usuario no se lo cambie).
 - **Who can view?**: marca **Admins y Users**.
4. **Save**.

Parte 2: Mapearlo al JWT vía Protocol Mapper

1. Menú izquierdo → **Clients** → click en piedrazul-frontend.
2. Pestaña **Client scopes**.
3. Click en piedrazul-frontend-dedicated.
4. Pestaña **Mappers** → **Configure a new mapper** → **By configuration** → busca y selecciona **User Attribute**.
5. Llena:
 - **Name**: usuario_id-mapper
 - **User Attribute**: usuario_id
 - **Token Claim Name**: usuario_id
 - **Claim JSON Type**: String
 - **Add to ID token**: ON

- **Add to access token:** ON ← este es el importante
- **Add to userinfo:** ON
- **Add to token introspection:** ON
- **Multivalued:** OFF

6. **Save.**

Parte 3: Setearlo para los usuarios de prueba ya creados

Parte 1: Crear el cliente confidential piedrazul-admin-cli

Este cliente lo usará SOLO el usuarios-service para llamar al Admin REST API de Keycloak. No lo usa el frontend.

1. Realm piedrazul → **Clients** → **Create client**.
2. **Step 1 — General settings:**
 - Client type: OpenID Connect
 - Client ID: piedrazul-admin-cli
 - Name: Pie Drazul Admin CLI (usuarios-service)
 - Description: Service account para crear usuarios via Admin REST API
 - **Next.**
3. **Step 2 — Capability config:**
 - **Client authentication:** ON ← clave, esto lo hace confidential.
 - **Authorization:** OFF.
 - **Authentication flow:**
 -  Standard flow
 -  Direct access grants
 -  Implicit flow
 -  **Service accounts roles** ← clave, esto habilita client_credentials grant.
 - **Next.**
4. **Step 3 — Login settings:** deja todo vacío. **Save.**

Parte 2: Asignar permisos al service account

1. Ya estás dentro del cliente piedrazul-admin-cli. Pestaña **Service accounts roles**.
2. Click **Assign role** → en el dropdown "**Filter by**" elige "**Filter by clients**".
3. Marca exactamente estas 4 roles del cliente realm-management (búscalas escribiendo "manage" o "view"):
 - manage-users
 - view-users
 - query-users
 - view-realm
4. **Assign**.

Parte 3: Copiar el client_secret

1. Pestaña **Credentials** del cliente piedrazul-admin-cli.
2. Verás un campo **Client secret** con un valor tipo xxx-xxx-xxx. Cópialo (botón del clip).
3. **Guárdalo**, lo vas a poner en usuarios-service/src/main/resources/application.properties cuando termine de programar (o me lo pasas y yo te indico dónde).

Parte 4: Crear el atributo persona_id (similar al usuario_id que ya hicimos)

1. **Realm settings** → pestaña **User profile** → **Create attribute**.
2. Llena:
 - **Attribute (Name)**: persona_id
 - **Display name**: Persona ID (dominio)
 - **Multivalued**: OFF
 - **Required**: OFF (los ADMIN/AGENDADOR podrían no tener persona vinculada)
 - **Permissions**:
 - **Who can edit?**: Admins.
 - **Who can view?**: Admins y Users.
3. **Save**.

Parte 5: Crear el protocol mapper para persona_id

1. **Clients** → piedrazul-frontend → pestaña **Client scopes**.
2. Click en piedrazul-frontend-dedicated.

3. Pestaña **Mappers** → **Configure a new mapper** → **By configuration** → **User Attribute**.

4. Llena:

- **Name:** persona_id-mapper
- **User Attribute:** persona_id
- **Token Claim Name:** persona_id
- **Claim JSON Type:** Long ← importante
- **Add to ID token:** ON
- **Add to access token:** ON
- **Add to userinfo:** ON

5. **Save.**